

Product Requirements Document (PRD)

FamScript Framework

Declarative Relationship Graph Framework

Mermaid-Native Compiler Architecture

1. Product Overview

Product Name

FamScript

Product Type

Declarative relationship graph framework that compiles structured relationship data into Mermaid-native visualizations.

Positioning Statement

“FamScript is a declarative relationship graph framework that compiles structured human relationship data into scalable Mermaid-native visualizations with validation, exportability, and framework interoperability.”

2. Core Philosophy

FamScript Source (.fam / .mmd)



FamScript Parser



Validation Engine



Relationship Graph Engine



Layout Engine



Mermaid Generator



SVG Renderer

↓
Export Adapters

Architectural Principles

- Frontend agnostic
 - Framework agnostic
 - Mermaid-native rendering
 - Validation-driven
 - Plugin-ready
 - Scalable compiler architecture
 - Runtime extensibility
 - Export-first ecosystem
-

3. Framework Goals

FamScript must provide:

- Human-readable relationship syntax
 - Compiler-based architecture
 - Mermaid-native rendering
 - Generation-aware layouts
 - Relationship validation
 - Export support
 - Runtime interactions
 - Plugin extensibility
 - Multi-framework support
 - Scalable graph rendering
-

4. Core Framework Scope

FamScript is not:

- A React-only library
- A static visualization component
- A DOM renderer
- A canvas-only renderer

FamScript is:

- A DSL (Domain Specific Language)
- A parser/compiler system

- A relationship graph engine
 - A layout engine
 - A Mermaid compiler
 - A framework ecosystem
-

5. Official Syntax Specification

5.1 Node Syntax

ID["Name | Age | Gender | Generation | Status"]

5.2 Examples

GROOM_SLF_001["Arjun Kumar | 28 | M | G0"]

F_SLF_001["Ramesh Kumar | 55 | M | G+1"]

GF_PAT_001["Shyam Lal | 80 | M | G+2"]

GGF_PAT_001["Ram Das | 104 | M | G+3 | deceased"]

UNK_MAT_001["Unknown | ? | ? | G+2"]

6. Relationship Syntax

Parent → Child

F_SLF_001 -->|"Father of"| GROOM_SLF_001

Marriage / Spouse

F_SLF_001 ---|"Married to"| M_SLF_001

Core Couple

GROOM_SLF_001 ==>|"Married to"| BRIDE_SLF_001

Distant / Step / Foster

A -.->

B

7. Generation Rules

Generation	Meaning
G+3	Great Grandparents
G+2	Grandparents
G+1	Parents / Uncles / Aunts
G0	Self / Bride / Groom
G-1	Children
G-2	Grandchildren

8. Core Design Principles

8.1 Central Anchoring

Primary individuals remain visually centered.

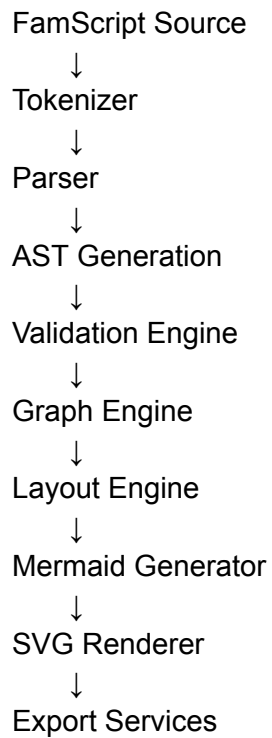
8.2 Generational Direction

- Older generations above
- Current generation centered
- Younger generations below

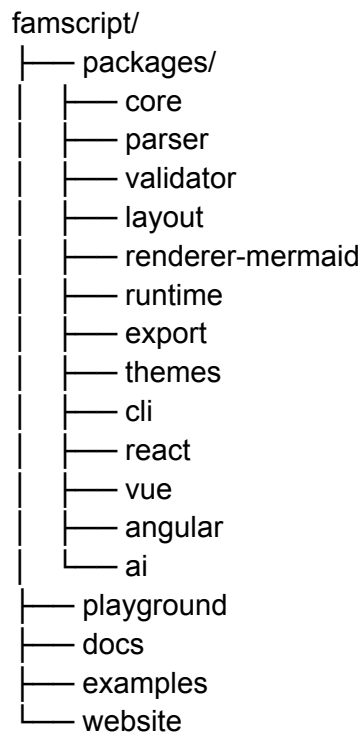
8.3 Layout Readability

- Minimized line crossing
 - Structured hierarchy
 - Stable spacing
 - Predictable layouts
 - Sibling clustering
 - Vertical lineage continuity
-

9. Compiler Pipeline



10. Monorepo Architecture



11. Core Packages

11.1 @famscript/core

Responsibilities

- Parse FamScript syntax
 - Build AST
 - Validate nodes
 - Validate edges
 - Detect cycles
 - Assign generations
 - Create graph model
 - Compile Mermaid output
-

11.2 @famscript/parser

Responsibilities

- Tokenization
- Grammar parsing
- AST generation
- Syntax analysis

Technology

Purpose	Technology
Tokenizer	Chevrotain
Parser	Chevrotain
AST	TypeScript interfaces

11.3 @famscript/validator

Responsibilities

Structural Validation

- Duplicate IDs
- Invalid edges
- Orphan nodes
- Cyclic ancestry
- Invalid generations

Semantic Validation

- Age rules
- Spouse parity
- Relationship legality
- Lineage validation

Custom Validators

```
validator.addRule({
  name: "royal-bloodline",
  validate(graph) {}
})
```

11.4 @famscript/layout

Responsibilities

- Generation layering
- Position calculation
- Spouse alignment
- Edge routing
- Adaptive spacing
- Subtree collapsing

Layout Rules

- Same generation = same row
- Spouse horizontal pairing
- Minimize edge crossing
- Lineage verticality
- Central anchoring
- Orphan branch isolation

Technologies

Purpose	Technology
Layout	elkjs
Graph positioning	dagre

11.5 @famscript/renderer-mermaid

Responsibilities

- AST → Mermaid conversion
 - Theme handling
 - Edge styling
 - Node styling
 - Subgraph optimization
 - Generation grouping
-

11.6 @famscript/runtime

Responsibilities

- Live updates
- Graph patching
- Zoom
- Pan
- Search
- Highlighting
- Relationship tracing

Runtime APIs

graph.highlightPath()

graph.expandBranch()

11.7 @famscript/export

Supported Formats

Format	Technology
SVG	Mermaid Native
PNG	sharp
PDF	Puppeteer

JSON	AST Export
Markdown	Mermaid Block
HTML	SSR Templates
XML	Genealogy Tools
GEDCOM	Interoperability

11.8 @famscript/themes

Responsibilities

- Theme creation
- Typography
- Spacing rules
- Generation styling
- Edge styling

Theme API

```
createTheme({  
  generation: {},  
  edges: {},  
  typography: {},  
  spacing: {}  
})
```

11.9 @famscript/cli

Commands

```
famscript render tree.fam
```

```
famscript validate tree.fam
```

```
famscript export tree.fam --pdf
```

```
famscript dev
```

```
famscript lint
```

12. Runtime Features

Node Interactions

- Hover cards
- Click expand
- Collapse subtree
- Search highlight
- Relation inspector

Navigation

- Zoom
- Pan
- Minimap
- Focus mode

Analytics

- Generation count
 - Lineage depth
 - Relation density
 - Branch complexity
-

13. Plugin System

Plugin API

```
createFamScript({  
  plugins: [  
    relationshipPlugin(),  
    marriagePlugin(),  
    timelinePlugin(),  
    analyticsPlugin()  
  ]  
})
```

Supported Plugin Types

- Timeline

- DNA
 - Ethnicity
 - Royal lineage
 - Inheritance mapping
 - Wedding maps
 - Clan systems
 - Religion lineage
 - Organization charts
-

14. Schema Specifications

Person Node Schema

```
interface PersonNode {  
  id: string  
  name: string  
  gender: string  
  generation: number  
  metadata: {}  
}
```

Relation Edge Schema

```
interface RelationEdge {  
  from: string  
  to: string  
  type: string  
}
```

Graph Schema

```
interface FamGraph {}
```

15. Validation Engine

Validation Gates

V1 — Age Gap Rule

parent.age $\geq$ child.age + 15

V2 — Spouse Generation Parity

Spouses must belong to the same generation.

V3 — Gender Role Consistency

- Father $\rightarrow$ M
- Mother $\rightarrow$ F
- Groom $\rightarrow$ M
- Bride $\rightarrow$ F

Unknown values use ?.

V4 — Duplicate Detection

No duplicate person within the same generation.

V5 — Circular Relation Guard

No person may become their own ancestor.

V6 — Sibling Parity

Siblings share generation levels.

V7 — In-Law Generation Rules

In-laws remain on spouse generation.

V8 — Cousin Parity

Cousins align to same generation layer.

16. Framework Integrations

Framework	Package
React	@famscript/react
Vue	@famscript/vue
Angular	@famscript/angular
Svelte	@famscript/svelte
Solid	@famscript/solid
Web Components	@famscript/web

17. Universal Web Component

Package

@famscript/web

Example

```
<fam-tree src="tree.fam"></fam-tree>
```

What is this?

18. Frontend Playground

Features

- Monaco editor
- Live Mermaid rendering
- Theme switching
- Export panel
- Validation console
- Live compile diagnostics

Technologies

Layer	Technology
Frontend	React
Framework	Next.js
Styling	Tailwind
Editor	Monaco Editor
UI	shadcn/ui

19. Documentation System

Layer	Technology
Docs	Docusaurus
MDX	MDX
Live code	Sandpack
Diagram docs	Mermaid

20. AI Features

Purpose	Technology
NLP	OpenAI API
Embeddings	pgvector
OCR	Tesseract
Image parsing	OpenCV
RAG	LangChain

21. Testing Stack

Purpose	Technology
Unit Tests	Vitest
E2E	Playwright
UI Tests	Testing Library
Visual Regression	Chromatic

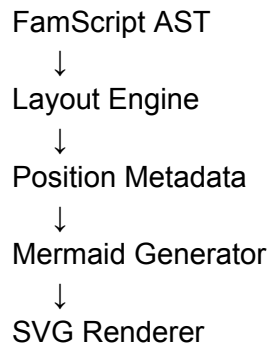
22. Build & Publishing

Purpose	Technology
Package Manager	pnpm
Monorepo	Turborepo
Bundling	Rollup
Build	tsup
Registry	npm
CI/CD	GitHub Actions

23. Official File Formats

Extension	Purpose
.fam	Native FamScript
.famconfig	Configuration
.famplugin	Plugin
.famcache	Compiled cache

24. Rendering Architecture



25. Technical Stack

Layer	Technology
Language	TypeScript
Parser	Chevrotain
Validation	Zod
Graph Engine	graphology
Layout	elkjs + dagre
Renderer	Mermaid.js
Frontend	React + Next.js
Editor	Monaco
State	Zustand
Export	Puppeteer + sharp
Docs	Docusaurus
Testing	Vitest + Playwright
Build	tsup
CI/CD	GitHub Actions

26. Graph Engine

Responsibilities

- Relationship graphs
- Hierarchy management
- Lineage traversal
- Branch collapsing
- DAG validation
- Cycle detection

Technologies

Purpose	Technology
Graph algorithms	graphology
Traversal	graphology
DAG validation	graphlib
Cycle detection	Custom DFS

27. Export System

Export Targets

- SVG
 - PNG
 - PDF
 - HTML
 - Markdown
 - JSON
 - XML
 - GEDCOM
-

28. Mermaid Rendering Strategy

Correct Architecture

FamScript Source → AST → Mermaid

Incorrect Architecture

FamScript → Direct DOM Rendering

Reason

Mermaid provides:

- Markdown compatibility
 - GitHub rendering
 - Export simplicity
 - Framework independence
 - Documentation interoperability
-

29. Supported Relationship Categories

Family

- Parent
- Child
- Sibling
- Grandparent
- Cousin
- Uncle
- Aunt
- In-laws
- Step relations
- Adoptive relations

Romantic

- Married
- Divorced
- Engaged
- Widowed
- Partnered

Social

- Friend
- Mentor
- Guardian

- Community member

Professional

- Manager
- Employee
- Advisor
- Co-founder

Hierarchical

- Royal lineage
- Clan systems
- Religious structures
- Organization charts

30. Competitive Advantage

Existing Tool	Problem
Mermaid	Generic diagrams only
D3	Too low-level
Graphviz	Poor genealogy UX
React Flow	Manual positioning
Neo4j visualizers	Not human-readable
GEDCOM tools	Outdated UI

31. FamScript Advantages

- Declarative syntax
- Human-readable
- Genealogy-aware
- Mermaid-native
- Framework-agnostic
- Validation-driven
- Exportable
- Plugin-ready

- Compiler architecture
 - Runtime extensibility
-

32. Most Important Architectural Decision

Keep separate:

- Parser
- Compiler
- Renderer
- Runtime
- UI

Correct Structure

FamScript Language



Compiler Core



Renderer Adapters



Frontend Adapters

This separation makes FamScript a true framework ecosystem instead of a UI library.

33. Final System Architecture

